

SAM-REF: Introducing Image-Prompt Synergy during Interaction for Detail Enhancement in the Segment Anything Model

Reproduction on a COCO subset

Master Project

at Friedrich-Alexander-Universität Erlangen-Nürnberg
at the Visual Computing
Cognitive Computer Vision Lab

Principal Supervisor:
Associate Supervisor:

Prof. Dr. Bernhard Egger
Prof. Andreas Kist

Author:

Zainab Sohail
zainab.z.sohail@fau.de
23040756

Submission:

26th March 2026

Abstract

Interactive segmentation systems refine an object mask based on user prompts such as clicks. Late fusion methods like the Segment Anything Model (SAM) are efficient because the image encoder runs once, but this design can limit fine detail recovery in later interaction. SAM-REF proposes a two stage refinement framework that keeps SAM's late fusion efficiency while reintroducing image prompt synergy through a lightweight refiner module placed after the SAM decoder. In this project, we reproduce the SAM-REF pipeline under reduced training data constraints. We train on a COCO subset of 500 training and 50 validation samples and evaluate on the standard interactive segmentation benchmarks like GrabCut, SBD, DAVIS and Berkeley.

Under this reduced data setting, SAM is the stronger baseline on average NoC across all benchmarks. StageA shows small improvement on GrabCut with ViTH, while StageB does not improve click efficiency and adds noticeable runtime overhead. In our project proposal we also planned to reproduce Table 2 (HQSeg), but we omit it here due to time and compute constraints and the additional HQSeg evaluation setup requirement.

Contents

1	Introduction	1
2	Method SAM-REF	2
2.1	Baseline: SAM as a late fusion interactive model	2
2.2	SAM-REF: Lightweight refiner for detail recovery	2
2.3	Two stage training strategy	2
3	Experimental Protocol and Setup	3
3.1	Computational Environment	3
3.2	Data: Training and evaluation sets	3
3.3	Caching SAM image embeddings	3
3.4	Interactive protocol and click simulation	3
3.5	StageA training	4
3.5.1	Loss function	4
3.5.2	Training hyperparameters	4
3.6	StageB training(Refiner module)	5
3.6.1	Loss function	5
3.6.2	Training hyperparameters	5
4	Evaluation	7
4.0.1	Quantitative results	7
4.0.2	Qualitative results	8
5	Conclusion	10
	References	III

Acronyms

AdamW Adaptive Moment Estimation with Weight Decay

COCO Common Objects in Context

DAVIS Densely Annotated Video Segmentation

DS Dynamic Selector

FN False Negative

FP False Positive

GFR Global Fusion Refiner

GT Ground Truth

LFR Local Fusion Refiner

NFL Normalized Focal Loss

NoC@90 average number of clicks required to reach $\text{IoU} \geq 90$

NoC@95 average number of clicks required to reach $\text{IoU} \geq 95$

ROI Region of Interest

ROIAlign Region of Interest Align

SAM Segment Anything Model

SAM-REF Segment Anything Model Refiner

SBD Semantic Boundaries Dataset

SPC Seconds Per Click

ViT Vision Transformer

1 Introduction

Interactive segmentation [1] aims to produce an accurate object mask using iterative user feedback, typically through positive and negative clicks. A key design choice is when to fuse image information with prompts. Early fusion systems such as FocalClick[1] and SimpleClick[7] jointly encode the image and prompts at each interaction, which can improve precision but increase latency because expensive image encoding is repeated. Late fusion systems extract image embeddings once and then combine them with prompts in a lightweight decoder for each interaction, achieving low latency.

Segment Anything Model (SAM) [5] is a strong example of late fusion: the image encoder is executed once, while the prompt encoder and mask decoder are lightweight. However, the decoupling of image encoding from later interactions can limit the model's ability to recover fine object details in challenging cases. Segment Anything Model Refiner (SAM-REF) [12] addresses this limitation by adding a lightweight refiner after the SAM decoder. The refiner directly fuses the original image with prompts during interaction to recover details, while also using SAM's embeddings to preserve semantic robustness.

In this project, we reproduce the SAM-REF [12] training and evaluation pipeline with constrained data and computation. Following the reproduction plan defined in the project proposal, we focus on (i) training the refinement stage using a reduced Common Objects in Context (COCO) [6] subset(500 train and 50 validation) and (ii) evaluating interactive performance on Densely Annotated Video Segmentation (DAVIS) [9], Semantic Boundaries Dataset (SBD) [2], GrabCut [10] and Berkeley[8] using average number of clicks required to reach $\text{IoU} \geq 90$ (NoC@90) and average number of clicks required to reach $\text{IoU} \geq 95$ (NoC@95). We also document the practical engineering required to run the late fusion interactive evaluation efficiently, including caching image embeddings as done in the original training strategy.

2 Method SAM-REF

2.1 Baseline: SAM as a late fusion interactive model

SAM [5] consists of three components: a heavy Vision Transformer (ViT) based image encoder, a lightweight prompt encoder and a mask decoder. During interaction, the image encoder is run once to produce image embeddings, and subsequent clicks are processed via the prompt encoder and decoder, making the interaction loop efficient.

2.2 SAM-REF: Lightweight refiner for detail recovery

SAM-REF [12] keeps the SAM pipeline for producing a coarse mask and then refines it using a decoupled lightweight refiner. The refiner is designed as a coarse to fine process with three main parts:

- Global Fusion Refiner (GFR): inspired by RITM [11], reintroduces image prompt synergy by building dense prompt maps and fusing them with the original image and up sampled encoder embeddings. This aims to recover low level boundary detail while preserving SAM’s semantics.
- Local Fusion Refiner (LFR): inspired by MaskRCNN [3], further refines difficult local regions by cropping a target area using Region of Interest Align (ROIALign) and pasting the refined local prediction back into the full mask.
- Dynamic Selector (DS): decides whether the local refinement should be pasted onto the current global prediction or onto the previous mask, to avoid overwriting already correct regions when the mask has mostly converged.

2.3 Two stage training strategy

SAM-REF uses a two stage strategy. In StageA, only the SAM mask decoder is fine tuned with simulated interaction while the image and prompt encoders remain frozen. In StageB, SAM remains frozen and a refiner network is trained to improve coarse SAM logits using dense click maps together with fused features. The refiner performs a global refinement at 256×256 resolution and a local Region of Interest (ROI) based refinement on difficult regions, followed by a simple selection step to avoid overwriting already correct regions.[12].

3 Experimental Protocol and Setup

3.1 Computational Environment

All experiments were conducted on a single NVIDIA GeForce GTX 1080 Ti (11 GB VRAM) on the FAU HPC cluster using PyTorch 2.6.0, Python 3.12, and CUDA. Stage A training completed in approximately 3 hours and Stage B in approximately 4 hours.

3.2 Data: Training and evaluation sets

We build a reduced COCO subset by iterating COCO instance annotations, filtering out crowd regions (iscrowd=1) and removing instances with area smaller than 50 pixels. Train/validation assignment is deterministic using a stable hash of the image id (seed 0, validation ratio 0.02). We cap the split to 500 unique training images and 50 unique validation images. This yields 2,849 training instances and 392 validation instances.

Following the standard splits used in prior work [13], we evaluated on GrabCut (50 instances), Berkeley (100 instances), DAVIS (345 instances) and SBD (345 instances) instead of full validation split, due to computational constraints.

3.3 Caching SAM image embeddings

To avoid repeatedly running the heavy SAM image encoder, we cache image embeddings separately and load them during training and evaluation. For each image, we apply SAM preprocessing (ResizeLongestSide to 1024 and padding to a 1024×1024 square) [5], run the image encoder once and store the embedding along with minimal metadata. Each cached embedding tensor has shape (1, 256, 64, 64).

For COCO training and validation, we precompute and store SAM image embeddings on disk and load them during optimization. For benchmark evaluation, we compute the embedding once per image and reuse it across clicks.

3.4 Interactive protocol and click simulation

We follow error driven click simulation following InterFormer’s [4] strategy as stated in SAM-REF [12]. The first click is placed at the distance transform center (argmax) of the largest connected component of the Ground Truth (GT) mask. For each subsequent step, we compute False Negative (FN) and False Positive (FP) between the current prediction and the GT, take the largest connected component in each and select the

larger of the two: an FN component generates a positive click, and an FP component generates a negative click. This “single largest error component per step” rule matches the InterFormer style interaction[4].

During training, the number of clicks per instance is sampled with normalized geometric decay proportional to γ^{k-1} where k is the click index with $\gamma = 0.6$, capped at 20 clicks and per click losses use the same decay weighting so earlier clicks contribute more. During evaluation, we simulate up to 20 clicks and stop early if no error region remains.

Clicks are represented as sparse point prompts (x, y, label) for SAM. For the refiner, we additionally build dense 256×256 click maps where each click is drawn as a filled disk of radius (5 pixels) at 256 resolution (obtained by downscaling the 1024 canvas by a factor of 4). We compute NoC@90 and NoC@95 from IoU using a logit prediction threshold > 0 (probability > 0.5) and a GT threshold of 0.5 [12].

3.5 StageA training

StageA fine tunes only the SAM mask decoder while keeping the image encoder and prompt encoder frozen. Clicks are simulated following the protocol in Section 3.3 (up to 20 per instance, geometric decay $\gamma = 0.6$). The decoder outputs 256×256 mask logits which are fed back as a mask prompt at the next click step [12].

3.5.1 Loss function

StageA is trained with Normalized Focal Loss (NFL) using focal exponent ($\gamma_{focal} = 2.0$). The loss is computed per click and aggregated with a geometric decay weight ($\gamma_{click} = 0.6$), so earlier clicks contribute more than later clicks.

3.5.2 Training hyperparameters

The model is optimized using Adaptive Moment Estimation with Weight Decay (AdamW) with $(\beta_1 = 0.9, \beta_2 = 0.999)$ and a weight decay of 0.01. The learning rate schedule begins with 16 warmup optimization updates followed by polynomial decay with power = 1.0, starting from an initial learning rate $= 1 \times 10^{-6}$ and peaking at $= 1 \times 10^{-4}$. Training uses a micro batch size of 4 with 2 gradient accumulation steps resulting in effective batch size of 8 and gradients are clipped to a maximum norm of 1.0, over a total of 1000 optimizer updates. Evaluation is performed every 200 updates on up to 200 validation instances, with logs recorded every 50 updates and checkpoints saved every 200 up-

dates. After training, the decoder state dictionary is saved as a checkpoint, which is used by StageB, to generate coarse masks during refiner training.

3.6 StageB training(Refiner module)

StageB trains a lightweight refiner while all SAM components remain frozen. At each click, SAM produces coarse mask logits at 256×256 . The refined mask logits from the previous click are fed back to SAM as a mask prompt when generating the next coarse logits (mask prompt feedback enabled). The refiner consumes dense maps formed by concatenating positive/negative disk maps (radius 5) and concatenated with the coarse mask converted to probabilities, together with aligned image features and upsampled SAM encoder embeddings and predicts refined mask logits and error maps. Local refinement is performed by forming an ROI around the largest changed region between the current and previous masks (union it with the last click and expand it by 1.4, applying ROIAlign) at 128×128 and pasting the refined ROI prediction back into the full 256×256 canvas. The simple error ratio heuristic (threshold 1.1) decides whether to paste onto the current global prediction or the previous mask to avoid overwriting converged regions.

3.6.1 Loss function

The per click objective combines supervision for both mask predictions and both error heads: $\lambda_{\text{err}}(L_{\text{err}}^G + L_{\text{err}}^L) + \lambda_{\text{mask}}(L_{\text{mask}}^G + L_{\text{mask}}^L)$ with weights $\lambda_{\text{err}} = 0.2$ and $\lambda_{\text{mask}} = 1.0$. The global and local mask losses each comprise three terms weighted normalized focal loss, BCE with error region weighting, and Dice loss, giving seven mask supervision terms in total. The mask terms upweight error pixels by $1.5\times$. The global and local error heads use BCE with logits + Dice to predict the coarse mask error map and its ROI cropped version.

3.6.2 Training hyperparameters

The model is optimized using AdamW with $(\beta_1 = 0.9, \beta_2 = 0.999)$ and a weight decay of 0.01. The learning rate schedule begins with 16 warmup optimization updates followed by polynomial decay with a power of 1.0, starting from an initial learning rate of $= 1 \times 10^{-7}$ and peaking at $= 5 \times 10^{-5}$. Training uses a micro batch size of 4 with 2 gradient accumulation steps resulting in effective batch size of 8 and gradients are clipped to a maximum norm of 1.0. The total number of optimization updates is set to 600, and reduced from 1000 after observing overfitting on the limited training set.

Evaluation is performed every 200 updates, with logs recorded every 50 updates and checkpoints saved every 200 updates and backpropagation is performed per click to reduce memory usage. After training, the refiner state dictionary is saved as a standalone checkpoint for evaluation.

4 Evaluation

We evaluate SAM, StageA, and StageB on DAVIS [9], SBD [2], GrabCut [10] and Berkeley[8], under the same click simulation policy (max 20 clicks) and report:

- NoC@90 and NoC@95: clicks required to reach IoU thresholds 0.90 and 0.95.
- SPC: seconds per click.

We report per click interaction latency Seconds Per Click (SPC) as the average wall clock time spent on the per click model update, measured around the forward pass that produces the new mask (and the refiner forward in StageB).

4.0.1 Quantitative results

Method	Train Data	Backbone	SPC/s	GrabCut		Berkeley		DAVIS		SBD	
				NoC90	NoC9	NoC90	NoC95	NoC90	NoC95	NoC90	NoC95
ViT-B											
SAM	COCO subset	ViT-B	0.0069	5.60	12.56	4.68	11.55	5.45	11.07	8.49	16.12
Stage-A	COCO subset	ViT-B	0.0069	7.04	14.94	7.06	15.17	10.24	17.08	10.63	17.12
Stage-B (Ours)	COCO subset	ViT-B	0.0202	6.30	14.70	7.23	15.62	10.43	17.27	10.94	16.86
ViT-H											
SAM	COCO subset	ViT-H	0.0088	4.46	11.38	4.53	10.32	5.18	10.07	8.09	15.68
Stage-A	COCO subset	ViT-H	0.0072	4.22	11.92	6.69	13.74	8.43	15.67	8.77	15.70
Stage-B (Ours)	COCO subset	ViT-H	0.0205	4.66	13.26	6.89	14.82	9.50	16.88	9.10	15.79

Table 1 Comparison of SPC, NoC90 and NoC95 on SAM, StageA and StageB. We report the results on the four mainstream datasets.

Based on our results, SAM is the strongest baseline. The only case where fine tuning improves over SAM is ViTH StageA on GrabCut, where NoC@90 decreases from 4.46 to 4.22 (NoC@95 increases slightly from 11.38 to 11.92). In all other settings, StageA and StageB require more clicks than SAM.

On Berkeley, both fine tuned models under perform SAM for both backbones. For ViTB, SAM achieves 4.68/11.55 while StageA reaches 7.06/15.17 and StageB 7.23/15.62. For ViTH, SAM achieves 4.53/10.32 while StageA reaches 6.69/13.74 and StageB 6.89/14.82. StageB is consistently worse than StageA on Berkeley.

The largest regressions appear on the harder benchmarks DAVIS and SBD. On DAVIS, ViTH StageB reaches NoC@90 of 9.50 compared to SAM’s 5.18, and ViTB StageB reaches 10.43 compared to SAM’s 5.45. On SBD, the same direction holds with a smaller gap (ViTH: 8.09 vs 9.10 at NoC@90, ViTB: 8.49 vs 10.94).

In terms of latency, SAM and StageA run at similar speed (ViTB: 0.0069 s/click, ViTH: 0.0088 vs 0.0072 s/click). StageB is slower due to the refiner and ROI processing, measuring around 0.020 s/click for both backbones. Since StageB does not improve click efficiency in this setting, the added runtime cost is not offset by better NoC.

4.0.2 Qualitative results

Figure 1 shows a GrabCut example (ViTH). After a strong negative click, SAM's IoU drops to 34.0% at click 5, while StageA and StageB remain stable at 95.4% and 95.2%. This is not visible in the averages of Table 1 but appears in individual cases. Figure 2 shows that SAM produces a large false positive region, whereas StageA and StageB keep mostly correct foreground coverage.

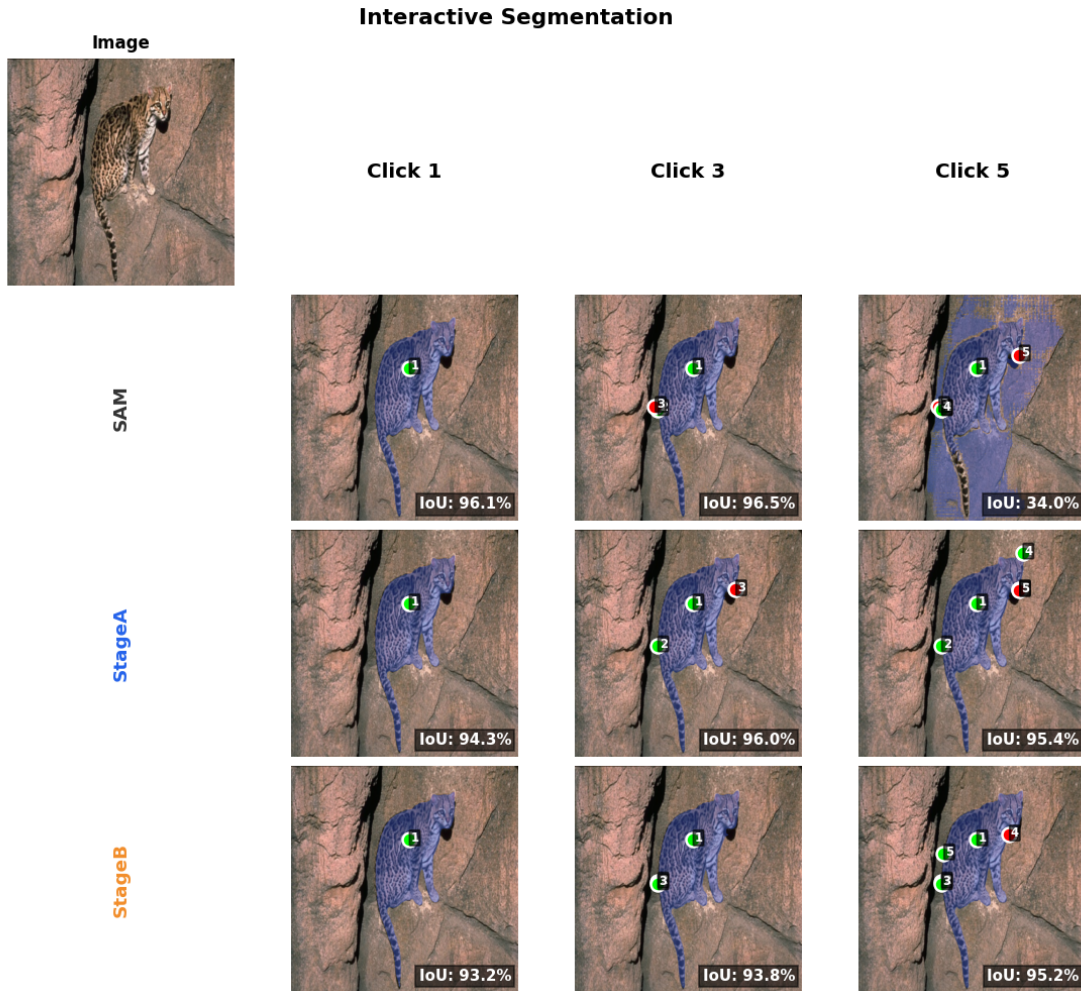


Figure 1 Click progression on a GrabCut sample (ViTH backbone). Rows: SAM, StageA, StageB. Columns: clicks 1, 3, 5. Blue overlay is predicted mask. Green dots are positive clicks, red dots are negative clicks and IoU is shown per cell.

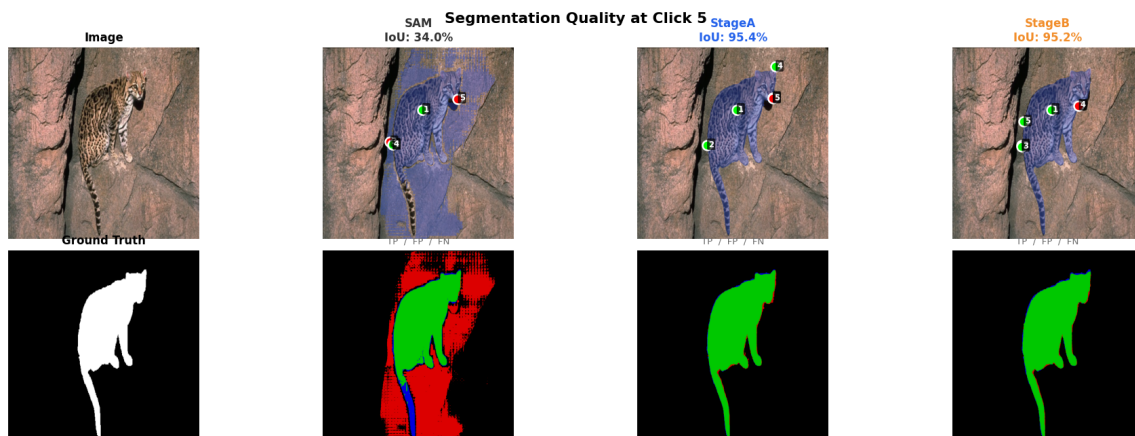


Figure 2 Error maps at click 5 for the same GrabCut sample. Top row: mask overlays with click locations. Bottom row: TP/FP/FN breakdown (green = TP, red = FP, blue = FN).

5 Conclusion

This reproduction evaluates SAM-REF under limited training data (500 COCO training images and 50 validation images). Under these constraints, the fine tuned models do not consistently improve over the frozen SAM baseline on GrabCut, Berkeley, DAVIS, and SBD.

StageA shows a small gain with ViTH on GrabCut, NoC@90 improves from 4.46 to 4.22. However, StageA regresses on the other datasets and on ViTB. StageB does not recover these losses and generally performs worse than StageA, especially on Berkeley and DAVIS.

Qualitative results provide additional context. In the GrabCut example in Figure 1, SAM's IoU collapses to 34.0% at click 5 after a strong negative click, while StageA and StageB remain stable at 95.4% and 95.2%. This failure does not appear in the NoC averages of Table 1, where the fine tuned models score worse than SAM overall, but the per sample results show the fine tuned decoder has learned more stable click by click behavior.

The runtime results further weaken the case for StageB in this setting. SAM and StageA stay below 0.01 s/click, while StageB increases latency to about 0.020 s/click. With higher NoC and higher SPC, StageB increases the total interaction cost without improving segmentation efficiency.

In summary, training on a 500 image COCO subset is not sufficient for StageB to generalise to the harder benchmarks. StageA with ViTH on GrabCut is the only setting with a measurable aggregate gain. On average NoC, SAM remains the stronger baseline across all four benchmarks. However, the qualitative results show that this does not tell the full story. SAM is more sensitive to individual negative clicks, and the fine tuned decoder handles these cases more stably. Scaling to the full COCO+LVIS dataset remains the necessary next step to assess whether StageB can reproduce the gains reported in the original paper.

References

- [1] Xi Chen et al. *FocalClick: Towards Practical Interactive Image Segmentation*. 2022. arXiv: 2204.02574 [cs.CV]. URL: <https://arxiv.org/abs/2204.02574>.
- [2] Bharath Hariharan et al. “Semantic Contours from Inverse Detectors”. In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2011, pp. 991–998. DOI: 10.1109/ICCV.2011.6126343. URL: <https://ieeexplore.ieee.org/document/6126343>.
- [3] Kaiming He et al. *Mask R-CNN*. 2018. arXiv: 1703.06870 [cs.CV]. URL: <https://arxiv.org/abs/1703.06870>.
- [4] You Huang et al. *InterFormer: Real-time Interactive Image Segmentation*. 2023. arXiv: 2304.02942 [cs.CV]. URL: <https://arxiv.org/abs/2304.02942>.
- [5] Alexander Kirillov et al. “Segment Anything”. In: *arXiv:2304.02643* (2023). URL: <https://arxiv.org/pdf/2304.02643>.
- [6] Tsung-Yi Lin et al. *Microsoft COCO: Common Objects in Context*. 2015. arXiv: 1405.0312 [cs.CV]. URL: <https://arxiv.org/abs/1405.0312>.
- [7] Qin Liu et al. “SimpleClick: Interactive Image Segmentation with Simple Vision Transformers”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2023, pp. 22290–22300. URL: <https://github.com/uncbiag/SimpleClick>.
- [8] Kevin McGuinness and Noel E. O’Connor. “A Comparative Evaluation of Interactive Segmentation Algorithms”. In: *Pattern Recognition Letters* 31.12 (2010), pp. 1625–1635. DOI: 10.1016/j.patrec.2010.05.002. URL: <https://www.sciencedirect.com/science/article/pii/S0167865510001315>.
- [9] Jordi Pont-Tuset et al. “The 2016 DAVIS Challenge on Video Object Segmentation”. In: *arXiv preprint arXiv:1611.00135* (2016). URL: <https://arxiv.org/abs/1611.00135>.
- [10] Carsten Rother, Vladimir Kolmogorov, and Andrew Blake. “GrabCut: Interactive Foreground Extraction using Iterated Graph Cuts”. In: *ACM SIGGRAPH 2004 Papers*. ACM, 2004, pp. 309–314. DOI: 10.1145/1186562.1015720.
- [11] Konstantin Sofiiuk, Iliia A. Petrov, and Anton Konushin. *Reviving Iterative Training with Mask Guidance for Interactive Segmentation*. 2021. arXiv: 2102.06583 [cs.CV]. URL: <https://arxiv.org/abs/2102.06583>.
- [12] Chongkai Yu et al. “SAM-REF: Introducing Image-Prompt Synergy during Interaction for Detail Enhancement in the Segment Anything Model”. In: *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2025, pp. 19356–19365. DOI: 10.1109/CVPR52734.2025.01803.
- [13] Chongkai Yu et al. *SAM-REF: Introducing Image-Prompt Synergy during Interaction for Detail Enhancement in the Segment Anything Model — Supplemental Material*. CVPR 2025 Supplemental. 2025. URL: https://openaccess.thecvf.com/content/CVPR2025/supplemental/Yu_SAM-REF_Introducing_Image-Prompt_CVPR_2025_supplemental.pdf.